ANDROID DEVELOPMENT

B.E. (Electronics & Telecommunication / Electronics) | 2019 Pattern | Semester VIII | Elective V

END SEMESTER EXAM — 17 May 2023 | Max Marks: 70

Q1 a) Explain basic building blocks of Android. [8 Marks]

Android applications are built around four fundamental components. These are the core building blocks that every Android app can consist of, and they must all be declared in the AndroidManifest.xml file.

1. Activities

An Activity represents a single screen with a user interface. It is the entry point for user interaction. When you open an app and see a screen — a login page, a home screen, or a settings page — each of those is an Activity. Every app must have at least one activity. Activities have a defined lifecycle managed by Android: they are created, started, resumed (active), paused, stopped, and eventually destroyed. The system calls specific methods at each transition.

2. Services

A Service runs in the background without any visible user interface. It is used for operations that need to continue running even when the user is not actively using the app. Typical examples include playing music while the user browses other apps, downloading files in the background, or syncing data with a server.

3. Broadcast Receivers

A Broadcast Receiver is a component that responds to system-wide broadcast announcements. Android sends broadcasts for many events: battery level changes, incoming calls, network connectivity changes, device boot completion, etc. Apps register to receive specific broadcasts and can react to them. For example, an app might listen for the 'network connected' broadcast to start syncing data.

4. Content Providers

A Content Provider manages a shared set of application data. It provides a standardized interface that allows other applications to query or modify the data, subject to access permissions. Android itself uses Content Providers extensively — the Contacts app, the Calendar app, and the Media Store all use Content Providers to make their data available to other apps.

Q1 b) Briefly explain Android API levels. [6 Marks]

An API Level is a unique integer value assigned to each version of the Android platform. Every time Google releases a new Android version with new features or changes, the API level is incremented. It is a way for developers and the system to agree on what capabilities are available.

How API Levels Are Used

- minSdkVersion — The lowest API level on which the app can be installed. Devices running older Android versions will not be able to install the app if their API level is below this value.
- targetSdkVersion — The API level the app was developed and tested against. Android may apply backward-compatible behaviors for apps targeting older API levels.
- compileSdkVersion — The API level used to compile the code. Set in build.gradle.

Android Version	API Level	Codename
Android 5.0	21	Lollipop
Android 6.0	23	Marshmallow
Android 7.0	24	Nougat

Android 8.0	26	Oreo
Android 9	28	Pie
Android 10	29	Q
Android 11	30	R
Android 12	31	S
Android 13	33	Tiramisu

Q1 c) What is Dalvik Virtual Machine? [4 Marks]

The Dalvik Virtual Machine (DVM) is a custom virtual machine created by Google specifically for running Android applications on mobile devices. It was the primary Android runtime until Android 4.4 (KitKat), after which it was gradually replaced by ART (Android Runtime).

- DVM is register-based (unlike the standard Java Virtual Machine which is stack-based), making it more efficient for low-memory mobile devices.
- It runs .dex (Dalvik Executable) files, not standard Java .class files. The dx tool converts .class files into the compact .dex format.
- Each Android app runs in its own DVM process, providing isolation and security.
- DVM was designed to run well with limited CPU speed, RAM, and battery life.
- It supports multiple virtual machine instances running simultaneously on a single device.

Q2 a) Explain components of communication in Android. [8 Marks]

Android provides several ways for components — activities, services, receivers — to communicate with each other, both within the same app and between different apps.

1. Intents

An Intent is a messaging object used to request an action from another component. It is the primary way components communicate in Android. An Explicit Intent specifies the exact component to start (used for in-app navigation). An Implicit Intent describes an action (like 'view this URL') and lets the system find a suitable component from any app to handle it.

2. Intent Filters

Intent Filters are declared in AndroidManifest.xml. They tell the Android system what types of implicit intents a component can handle. When an implicit intent is fired, Android looks through all installed apps' intent filters to find a match.

3. Broadcast and Broadcast Receivers

Android can send broadcasts for system events (battery low, network changed, etc.) or apps can send custom broadcasts. Broadcast Receivers listen for these messages. When a matching broadcast is detected, the receiver's onReceive() method is called. Receivers can be registered statically in the manifest or dynamically in code.

4. Content Providers

Content Providers allow structured data to be shared between apps. Data is accessed through a URI and the ContentResolver class. This enables secure, standardized access to data like contacts, images, or any custom database.

5. Pending Intents

A PendingIntent is a token that allows another application (like the notification system or an alarm manager) to execute a predefined Intent on behalf of your app at a future time. Used in notifications and alarms.

Q2 b) List out UI Components. [4 Marks]

Android UI components (also called Views or Widgets) are the visual elements used to build the user interface. They are organized into the following categories:

Text Components

- TextView — Displays non-editable text on screen.
- EditText — An input field where users can type text.
- AutoCompleteTextView — Shows typing suggestions in a dropdown.

Button and Selection Components

- Button — Standard clickable button.
- ImageButton — Button that displays an image.
- CheckBox — A toggle that can be checked or unchecked.
- RadioButton / RadioGroup — Single-choice selection among multiple options.
- Switch / ToggleButton — On/off toggle controls.

Image and Media Components

- ImageView — Displays an image.
- VideoView — Plays video content.

Progress and Feedback Components

- ProgressBar — Shows circular or horizontal loading progress.
- SeekBar — A draggable slider for selecting a value from a range.

- RatingBar — Displays and lets users set a star rating.

List and Grid Components

- ListView — Vertical scrollable list of items.
- GridView — Two-dimensional grid of scrollable items.
- Spinner — Dropdown selection widget.
- RecyclerView — Efficient, flexible list/grid (modern replacement for ListView).

Date and Time Components

- DatePicker — UI for selecting a date.
- TimePicker — UI for selecting a time.
- Chronometer — Count-up timer display.

Q2 c) Explain: i) Assets ii) Layouts iii) Drawable Resource. [6 Marks]

i) Assets

The assets/ folder in an Android project is used to store raw files that need to be accessed exactly as they are — without any processing or compilation by the build system. Unlike the res/ folder, files in assets/ are not compiled or compressed. They keep their original file names and directory structure. You access them programmatically using the AssetManager class through getAssets().open('filename'). Common uses include HTML files for a WebView, custom font files (.ttf, .otf), configuration files in JSON or XML format, and game data files.

ii) Layouts

Layouts are XML files stored in the res/layout/ folder that define the visual structure of a user interface screen or component. Each layout file describes how views (buttons, text fields, images) are arranged on the screen. Layouts use ViewGroups as containers. Common layout types include LinearLayout (arranges children in a single row or column), RelativeLayout (positions children relative to each other or the parent), ConstraintLayout (positions elements using flexible constraints — the most powerful and recommended), FrameLayout (stacks children on top of each other — used for fragments), and TableLayout (arranges children in rows and columns).

iii) Drawable Resource

Drawable resources are stored in the res/drawable/ folder and represent visual elements that can be drawn on screen. They include: Bitmap Drawables — image files in PNG, JPEG, GIF, or WebP format. Shape Drawables — XML-defined geometric shapes like rectangles, ovals, and lines, with specified colors, gradients, and corners. Layer-List Drawables — multiple drawables stacked in layers. State-List Drawables — different drawables for different states (normal, pressed, focused, disabled) used for buttons. Transition Drawables — for animated transitions between two drawables. Drawables are referenced in layouts using @drawable/filename.

Q3 a) What are activities and explain what is a lifecycle? [7 Marks]

What is an Activity?

An Activity is a fundamental Android component that provides a single screen with which the user can interact. It represents one focused thing the user can do, such as viewing a list of emails, composing a new email, or reading a specific email. Each activity is independent, has its own layout, and is declared in AndroidManifest.xml.

Activity Lifecycle

Android manages activities through a well-defined lifecycle. As the user navigates the app and the system needs resources, activities transition between different states. Android calls specific callback methods at each transition so that your app can respond appropriately — saving data, releasing resources, or resuming work.

Callback Method	When It Is Called	What To Do Here
onCreate()	When the activity is first created	Initialize UI, set layout with setContentView(), get references to views
onStart()	When the activity becomes visible to the user	Start animations or other visual processes
onResume()	When the activity is in the foreground and interactive	Start playing media, register sensors, resume any paused work
onPause()	When another activity takes focus (partially visible)	Pause media, release sensors, save unsaved data
onStop()	When the activity is no longer visible	Release heavy resources, save app state

onRestart()	When the activity is coming back from stopped state	Restore any resources released in onStop()
onDestroy()	Before the activity is destroyed completely	Release all remaining resources and clean up

Understanding the lifecycle is crucial for writing apps that don't leak memory, don't waste battery, and preserve user data across transitions like screen rotation, incoming calls, or switching between apps.

Q3 b) Explain i) Components of a screen ii) Adapting to Display Orientation. [10 Marks]

i) Components of a Screen

The Android screen UI consists of:

- View — The fundamental building block. Every visible element (buttons, text, images) is a View. Views occupy a rectangular area and handle drawing and touch events.
- ViewGroup (Layouts) — Special Views that contain other Views as children and define how they are arranged. Examples: LinearLayout, RelativeLayout, ConstraintLayout, FrameLayout.
- Window — Every Activity has a single Window that holds the entire view hierarchy. The Window occupies the whole screen and provides the drawing surface.
- Surface — A raw drawing area used by the window to render content. The SurfaceView class provides direct access to this for high-performance rendering like games and camera.
- Canvas — Used for custom 2D drawing. The Canvas object provides methods like drawCircle(), drawRect(), drawText(), and drawBitmap(). You use it inside a custom View by overriding onDraw().
- Common UI components: TextView, EditText, Button, ImageView, CheckBox, RadioButton, SeekBar, ProgressBar, ListView, GridView, Spinner, DatePicker, TimePicker.

ii) Adapting to Display Orientation

Android devices can be used in portrait (vertical, taller than wide) or landscape (horizontal, wider than tall) orientation. A well-designed app must work correctly in both. There are several strategies:

- Lock Orientation — In AndroidManifest.xml, set android:screenOrientation='portrait' or 'landscape' on the activity. Simple but not user-friendly.
- Alternate Layout Resources — Create a res/layout-land/ folder and place a landscape-optimized version of the layout there with the same filename. Android automatically selects the right file based on current orientation.
- Handle in Code — Add android:configChanges='orientation|screenSize' to the activity in the manifest. This prevents Android from restarting the activity on rotation. Instead, it calls onConfigurationChanged() where you can adjust the UI programmatically.
- Save and Restore State — When the activity is restarted due to rotation, override onSaveInstanceState() to save important data into a Bundle, and restore it in onCreate() or onRestoreInstanceState().

Q4 a) What is Intents? Explain Linking activities. [7 Marks]

What is an Intent?

An Intent is a messaging object in Android used to request an action from another component of the same app or even from a different app. Intents are the primary way that Android components communicate and cooperate with each other.

There are two types: An Explicit Intent explicitly specifies the target component by its class name. This is used when you want to open a specific activity within your own app. An Implicit Intent does not name a specific target. Instead, it declares what action needs to be done (like opening a browser, sharing text, or taking a photo) and Android finds the right app to handle it.

Intents can also carry data using putExtra() with key-value pairs, and they can have flags and categories that further describe the request.

Linking Activities Using Intents

To navigate from one activity to another, you create an Intent object with the source context and the destination activity class. Then call startActivity(intent). If you need data back from the destination, use startActivityForResult() instead. Pass extra data to the destination using intent.putExtra('key', value). In the destination activity, retrieve the sent data using getIntent().getStringExtra('key') or

similar methods. If using startActivityForResult, the destination activity calls setResult() with the data before finishing, and the source activity receives it in onActivityResult().

Q4 b) Explain i) Fragment Lifecycle ii) Split/Multi screen Activities. [10 Marks]

i) Fragment Lifecycle

A Fragment is a modular portion of UI within an Activity. It has its own lifecycle that is closely tied to the host Activity's lifecycle. When the Activity is paused, all fragments in it are also paused.

Callback Method	When Called
onAttach()	Fragment is attached to its host Activity
onCreate()	Fragment is being created — initialize non-UI components
onCreateView()	Create and return the fragment's view hierarchy (inflate layout)
onViewCreated()	View is fully created — bind view references here
onStart()	Fragment becomes visible
onResume()	Fragment is active and interactive
onPause()	Fragment is partially hidden or losing focus
onStop()	Fragment is no longer visible
onDestroyView()	Fragment's view is being removed
onDestroy()	Fragment is being destroyed — release resources
onDetach()	Fragment is detached from its host Activity

ii) Split / Multi-Screen Activities

Multi-screen (split-screen) Activities allow two activities or fragments to be displayed side by side on larger screens like tablets, or vertically stacked in multi-window mode on phones. This makes better use of available screen space.

The most common pattern is the Master-Detail pattern: A list (master) is shown on the left and when an item is selected, its details appear on the right. On phones, these are two separate activities (full screen each). On tablets, both are shown in the same activity using two fragments.

Android 7.0 (Nougat) introduced system-level multi-window support, allowing users to manually put two apps side by side. Apps can declare android:resizeableActivity='true' in the manifest to support this. In Android 12 and later, split-screen is even more fluid and easier to use.

Q5 a) Explain following user interfaces: i) Text view ii) Progress bar View. [10 Marks]

i) TextView

TextView is the most basic and widely used UI component in Android. It is used to display text on the screen that the user can read but typically cannot edit directly (for editing, EditText is used, which is a subclass of TextView).

Key attributes include text (sets the displayed text), textSize (sets font size in sp units), textColor (sets the color), textStyle (bold, italic, or normal), gravity (aligns text within the view — start, center, end), maxLines (limits how many lines are shown), ellipsize (controls how overflowing text is cut — usually with '...'), and autoLink (automatically makes URLs, phone numbers, and email addresses into clickable links).

In Java code, you get a reference to the TextView using findViewById(), then call methods like setText() to change the text, setTextColor() to change color, and setTextSize() to change size at runtime.

TextView also supports Spannable text, which allows different portions of the same text to have different styles, colors, sizes, or even be clickable links — all without using multiple TextViews.

ii) ProgressBar View

ProgressBar is a UI widget used to indicate that some work is in progress or to show how much of a task has been completed. It gives users visual feedback that the app is working and hasn't frozen.

There are two main styles: Indeterminate ProgressBar — shows a continuously spinning or pulsing animation when you don't know how long the task will take (like connecting to the internet). It communicates 'please wait' without a specific completion percentage. Determinate ProgressBar — shows a horizontal bar that fills up from 0% to 100% as the task progresses. Used when you know the total amount of work, such as a file download progress.

Key attributes include style (set to @style/Widget.AppCompat.ProgressBar for circular spinning, or @style/Widget.AppCompat.ProgressBar.Horizontal for a horizontal bar), max (the maximum value, typically 100), progress (the current value), and indeterminate (true or false).

In Java, set the current progress using progressBar.setProgress(value) and show/hide it using progressBar.setVisibility(View.VISIBLE) or View.GONE.

Q5 b) Explain Animation in Android. [8 Marks]

Android provides three animation systems for creating visual effects and motion in applications:

1. View Animation (Tween Animation)

This is the original animation system. It applies transformations to a View's appearance — such as moving it across the screen (translate), spinning it (rotate), resizing it (scale), or fading it in or out (alpha). Animations are defined in XML files in the res/anim/ folder. You load the animation using AnimationUtils.loadAnimation() and apply it to a view using view.startAnimation(). An important limitation is that while the view appears to move, its actual touchable area and real position stay at the original location.

2. Property Animation (Animator)

Introduced in API 11, this is a more powerful system that actually changes the real property values of objects, not just their appearance. The ObjectAnimator class animates specific properties like translationX (horizontal position), alpha (transparency), rotation, scaleX, and scaleY. An AnimatorSet allows multiple property animations to run together or in a defined sequence. Since it changes actual properties, the view's touch area moves with it. This system works on any Java object, not just Android Views.

3. Drawable Animation (Frame Animation)

This works like a traditional animated GIF or a film reel. A sequence of drawable images is defined in an XML file in res/drawable/ as an AnimationDrawable. Each frame specifies a drawable and how long it should be displayed. The animation is set as the background of an ImageView. When

started, it plays through all the frames in order, creating the illusion of movement. The oneshot attribute controls whether it plays once or loops continuously.

Q6 a) Briefly explain Android Menus. [10 Marks]

Android provides three types of menus for presenting actions and options to users:

1. Options Menu

The Options Menu is the primary collection of menu items for an activity. It appears when the user taps the three-dot overflow icon in the top right of the action bar (toolbar). It is best used for actions that affect the entire screen — such as Search, Settings, Refresh, or Help.

Implementation: Create a menu XML file in res/menu/. In the activity, override onCreateOptionsMenu() to inflate the menu file using getMenuInflater().inflate(). Override onOptionsItemSelected() to handle item clicks based on item ID.

2. Context Menu

A Context Menu appears when the user performs a long press on a UI element like a list item, an image, or any other view. It provides actions specific to that element, such as Copy, Delete, Edit, or Share.

Implementation: Call registerForContextMenu(view) to register which view should show the context menu. Override onCreateContextMenu() to add menu items. Override onContextItemSelected() to handle the user's selection.

3. Popup Menu

A Popup Menu is a modal dropdown menu anchored to a specific view on the screen. It appears just below (or above) the anchor view when triggered. It is used for actions that are related to a specific UI element but aren't important enough for the action bar.

Implementation: Create a PopupMenu object with the context and anchor view. Inflate a menu XML file. Set an OnMenuItemClickListener. Call popup.show() to display it.

Using Submenus

All three menu types support nested submenus — a menu item that opens a secondary menu when selected. Submenus are created by grouping items under a parent item in the menu XML.

Q6 b) Explain Graphics in Android. [8 Marks]

Android supports both 2D and 3D graphics for creating visual content in applications.

2D Graphics

2D graphics in Android can be created in several ways:

- Canvas Drawing — The most flexible approach. You create a custom View by extending the View class and overriding the onDraw(Canvas canvas) method. The Canvas object provides methods to draw shapes (circles, rectangles, lines, arcs), text, and bitmaps. The Paint object controls visual properties like color, stroke width, text size, anti-aliasing, and whether shapes are filled or just outlined.
- XML Drawables — Shape drawables defined in XML allow you to create geometric shapes (rectangles, ovals, lines) with colors, gradients, borders, and rounded corners without any image files. These are lightweight and scale perfectly to any size.
- Bitmap Graphics — PNG, JPEG, or WebP image files placed in the res/drawable/ folder. Loaded as BitmapDrawable objects and displayed in ImageViews or drawn directly on a Canvas.
- SurfaceView — For continuously updating graphics like games or live camera previews. Provides a dedicated drawing thread separate from the main UI thread for smooth performance.

3D Graphics

3D graphics are achieved using OpenGL ES (Open Graphics Library for Embedded Systems), which is a subset of OpenGL designed for mobile and embedded devices. Android supports OpenGL ES 1.0, 2.0, and 3.0.

- GLSurfaceView — A special view that manages an OpenGL rendering context and provides a surface to draw 3D graphics on. It handles the creation of the OpenGL context and manages the background rendering thread.
- GLSurfaceView.Renderer — An interface that defines the actual rendering logic. You implement three methods: onSurfaceCreated() for initial setup, onSurfaceChanged() when the view size changes, and onDrawFrame() which is called repeatedly to draw each frame.

Q7 a) Explain Internal and external storage. [7 Marks]

Internal Storage

Internal storage refers to the device's built-in memory that is reserved for app-specific private files. Files written to internal storage are private to the app — other apps cannot access them and the user cannot browse them without root access. These files are automatically deleted when the app is uninstalled.

No special permissions are required to use internal storage. You write files using openFileOutput() which returns a FileOutputStream. The mode Context.MODE_PRIVATE creates a new file or overwrites an existing one. Context.MODE_APPEND adds to an existing file. You read files using openFileInput() which returns a FileInputStream. You can list all files in internal storage using fileList() and delete them using deleteFile('filename').

External Storage

External storage is a shared storage area accessible to the user and potentially to other apps. It can be the device's built-in external partition or an SD card. Files here persist even after the app is uninstalled unless saved to the app-specific external directory (getExternalFilesDir()).

For Android versions below Android 10 (API 29), the READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE permissions must be declared in the manifest, and runtime permission must be requested from the user. From Android 10 onwards, scoped storage restrictions apply and most external storage access happens through the MediaStore API or Storage Access Framework.

Before writing, check if external storage is available using Environment.getExternalStorageState() and verify it returns MEDIA_MOUNTED. Use getExternalFilesDir() to get the app-specific external directory which doesn't require special permissions. Standard Java File I/O (FileWriter, BufferedReader) is used to read and write files.

Aspect	Internal Storage	External Storage
Access	Private to app only	Shared with other apps and user
Permission needed	None	Required for Android < 10
Deleted on uninstall	Yes	Only app-specific folder
Always available	Yes	No — SD card may be absent

Q7 b) Write a short note on SQLite. [10 Marks]

SQLite is a lightweight, serverless, open-source relational database management system that is embedded directly into the Android operating system. It does not require a separate database server to be running — the entire database is stored as a single file on the device. SQLite uses standard SQL syntax and is ACID-compliant, meaning it guarantees data integrity.

Features of SQLite in Android

- The database is stored as a file at /data/data/packagename/databases/. It is private to the app.
- Supports all standard SQL operations: CREATE TABLE, INSERT INTO, SELECT, UPDATE, DELETE.
- Supports multiple data types: INTEGER, TEXT, REAL (floating point), BLOB (binary data), NULL.
- No separate server process — the database engine runs within the app's process itself.
- Very small memory footprint — suitable for mobile devices.

SQLiteOpenHelper

The primary class for working with SQLite in Android is SQLiteOpenHelper. You create a subclass of it. The constructor takes the context, database name, optional cursor factory, and version number. The onCreate() method is called automatically when the database is created for the first time — you write your CREATE TABLE statements here. The onUpgrade() method is called when you increase the version number — use it to add new tables or columns without losing existing data.

Database Operations

Obtain an SQLiteDatabase instance using getWritableDatabase() for read-write operations or getReadableDatabase() for read-only. Insert data using ContentValues with db.insert(). Update data using db.update() with a WHERE clause. Delete data using db.delete(). Read data using db.rawQuery() which returns a Cursor. The Cursor is like a pointer to the result set — use moveToFirst() and moveToNext() to iterate through rows. Always close the Cursor and the database when done to free resources.

Alternatives for Simple Data

SQLite is appropriate for structured data with multiple fields and relationships. For simple key-value settings, SharedPreferences is simpler. For large files like images or audio, file storage is more appropriate.

Q8 a) Explain database in Android. [7 Marks]

Android provides several mechanisms for storing data persistently. For structured relational data, SQLite is the primary choice, but there are other options as well.

SQLite Database

SQLite is the built-in database engine in Android. It is used to store structured, relational data that needs to be queried, filtered, or sorted. It supports all standard SQL operations and stores data in tables with rows and columns. It is accessed through the SQLiteDatabase class and managed using SQLiteOpenHelper. Suitable for storing user data, app content, cached API data, and any structured information.

Room Database (Modern Approach)

Room is an abstraction layer built on top of SQLite, provided as part of Android Jetpack. It simplifies database code by using annotations to define tables (@Entity), data access objects (@Dao), and the database itself (@Database). Room validates SQL at compile time, preventing runtime errors. It works well with LiveData and ViewModel for a modern architecture.

SharedPreferences

SharedPreferences is used for storing simple key-value pairs of primitive data types (booleans, strings, integers, floats, longs). It is stored as an XML file in the app's private storage. Not suitable for complex structured data, but perfect for user settings, login state, or app preferences.

Key Database Terms

Term	Meaning
Table	A structured set of data organized in rows and columns
Row / Record	A single entry in a table
Column / Field	A specific attribute of the data (like name or age)
Primary Key	A unique identifier for each row (usually an auto-incremented integer)
Cursor	An object that points to the result of a database query and lets you iterate through rows
ContentValues	A key-value map used to specify the data to insert or update

Q8 b) Short note on: i) Google Maps ii) Monitoring a Location. [10 Marks]

i) Google Maps

The Google Maps Android API allows developers to embed fully interactive Google Maps directly inside Android applications. Users can view maps, zoom in and out with pinch gestures, pan by swiping, switch between map types, and interact with markers and overlays.

To use Google Maps, you add the Google Play Services Maps dependency in build.gradle and obtain an API key from the Google Cloud Console. The API key is added to AndroidManifest.xml inside a meta-data tag.

In your layout, add a SupportMapFragment which provides the map view. In the Activity, implement the OnMapReadyCallback interface. The onMapReady(GoogleMap googleMap) callback gives you the GoogleMap object. With it, you can add markers at specific coordinates using MarkerOptions (with position, title, and snippet), move and animate the camera to a location using CameraUpdateFactory.newLatLngZoom(), draw polylines for routes, and draw circles or polygons.

Map types available: MAP_TYPE_NORMAL (standard road map), MAP_TYPE_SATELLITE (aerial imagery), MAP_TYPE_HYBRID (satellite + roads overlay), and MAP_TYPE_TERRAIN (topographic map).

ii) Monitoring a Location

Location monitoring means continuously receiving updates about the device's geographic position as it moves. Android provides two approaches: the older LocationManager API and the newer, recommended Fused Location Provider API from Google Play Services.

With LocationManager, you need ACCESS_FINE_LOCATION (for GPS) and ACCESS_COARSE_LOCATION (for network) permissions. At runtime on Android 6.0+, you must request these permissions from the user. You create a LocationListener with the onLocationChanged() callback that receives a Location object whenever position changes. You register it with requestLocationUpdates() specifying the provider, minimum time between updates (in milliseconds), and minimum distance change (in meters). Always call removeUpdates() in onPause() or onStop() to stop receiving updates and conserve battery.

The Fused Location Provider is preferred in modern apps. It automatically blends GPS, Wi-Fi, and network signals to provide the best accuracy with the least battery drain. It uses LocationRequest to specify accuracy and update interval requirements, and LocationCallback to receive updates.